Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА ИУ7 «ИНФОРМАЦИОННОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Разработка базы данных для сайта-блога с справочником растений для ландшафтных архитекторов

Студент	**ИУ7-62Б**		
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсового проекта			**Гаврилова Ю.М.**
		(подпись, дата)	(И.О. Фамилия)
Консультант			
		(подпись, дата)	(И.О. Фамилия)

2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой ИУ7
(индекс)
И. В. Рудаков
(И.О. Фамилия)
(подпись) *28.02.2025 года*
(дата)

З А Д А Н И Е
на выполнение курсовой работы

по дисциплине **«Базы данных»**

Студент группы **ИУ7-62Б Шахнович Дмитрий Сергеевич**
(Фамилия, имя, отчество)

Тема курсовой работы **Разработка базы данных для сайта-блога с справочником растений для ландшафтных архитекторов**

Направленность КР (учебная, исследовательская, практическая, др.) **учебная**

Источник тематики (кафедра, предприятие, НИР) **кафедра**

Задание

Провести анализ предметной области ландшафтного дизайна, в частности использования баз данных растений в ландшафтном дизайне. Сформулировать описание пользователей сайта-блога для доступа к базе данных. Спроектировать сущности базы данных растений и сайта-блога. Спроектировать процедуру проверки свойств у растения, присущих его категории. Выбрать средства реализации базы данных. Реализовать разработанные сущности растений и сайта. Исследование времени выполнения запросов получения данных в зависимости от способа представления растений в базе данных.

Оформление курсовой работы:

Расчетно-пояснительная записка на 18-40 листах формата А4. Презентация к курсовой работе на 6-15 слайдах.

Дата выдачи задания «28» февраля 2025 г.

Руководитель курсовой работы _____ **Гаврилова Ю.М.**
(подпись, дата) (И.О. Фамилия)

Студент _____ _____
(подпись, дата) (И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсовой работы

по дисциплине **«Базы данных»**

Студент группы **ИУ7-62Б Шахнович Дмитрий Сергеевич**

(Фамилия, имя, отчество)

Тема курсовой работы

Разработка базы данных для сайта-блога с справочником растений для ландшафтных архитекторов

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель КП	Куратор
1.	Задание на выполнение курсовой работы			*Гаврилова Ю.М.*	
2.	1 модуль	_____ *Планируемая дата*		*Гаврилова Ю.М.*	
3.	2 модуль	_____ *Планируемая дата*		*Гаврилова Ю.М.*	
4.	Оформление РПЗ (Отчета)	_____ *Планируемая дата*		*Гаврилова Ю.М.*	
5.	Подготовка доклада и презентации (при необходимости)	_____ *Планируемая дата*		*Гаврилова Ю.М.*	
6.	Защита курсовой работы	_____ *Планируемая дата*		*Гаврилова Ю.М.*	

Студент _____ Руководитель работы _____
(подпись, дата) (подпись, дата)

РЕФЕРАТ

Курсовая работа 53 с., 10 рис., 19 ист., 7 табл., 2 прил.

Ключевые слова: СУБД, БД, Postgresql, minIO, Go, ландшафтная архитектура

Цель работы: разработка базы данных для сайта-блога ландшафтного архитектора.

Результаты работы:

— проведён анализ предметной области и рассмотрены известные решения;

— формализованы данные и роли пользователей;

— спроектирована и разработана база данных, описаны её сущности и связи;

— спроектирован и разработан триггер проверки свойств растений;

— реализован сайт-блог, использующий базу данных;

— проведено исследование зависимости времени выполнения запроса выборки растений от их представления в базе данных.

Исследование показало, что хранение характеристик растений в виде JSON позволяет ускорить время запроса в 3-5 раз по сравнению с моделью EAV.

Содержание

РЕФЕРАТ **4**

ВВЕДЕНИЕ **7**

1 Аналитическая часть **8**
1.1 Готовые решения 8
1.1.1 Сайт PlantsFinder 8
1.1.2 Сайт Catalog.greenmaster.by 9
1.1.3 Сайт Ideas.by 10
1.1.4 Сравнение 10
1.2 Требование к разрабатываемому приложению 11
1.3 Роли пользователей 11
1.4 Формализация данных 12
1.5 Выбор базы данных 15

2 Конструкторская часть **17**
2.1 Схема базы данных 17
2.2 Ограничения целостности 18
2.3 Триггер проверки свойств растения 18
2.4 Ролевая модель 19

3 Технологическая часть **21**
3.1 Средства реализации 21
3.2 Создание базы данных 21
3.3 Роли и их права 22
3.4 Триггер проверки свойств растений 23
3.5 Сайт-блог 23
3.6 Тестирование 23
3.6.1 Тестирование приложения 23
3.6.2 Тестирование триггера проверки характеристик растений 24

4 Исследовательская часть **26**
4.1 Цель исследования 26
4.2 Описание исследования 26
4.3 Результаты исследования 28

ЗАКЛЮЧЕНИЕ **31**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **32**

Приложение А **34**

Приложение Б **53**

ВВЕДЕНИЕ

В настоящее время в сфере разработки ландшафтной архитектуры используются информационные технологии, в частности онлайн-платформы, которые используются для поиска и хранения информации или справочных материалов по ландшафтной архитектуре. Использование таких технологий позволяет ландшафтным архитекторам ускорить разработку проектов, сосредоточившись на проектировании, а не ручном поиске информации [1].

Целью данной работы является разработка базы данных для сайта-блога ландшафтного архитектора со справочником растений.

Для достижения поставленной цели необходимо решить следующие задачи:

1) проанализировать предметной области и рассмотреть известные решения;
2) формализовать данные и выделить роли пользователей базы данных;
3) спроектировать базу данных, описать ее сущности и связи, спроектировать триггер проверки свойств растений;
4) выбрать систему управления базами данных;
5) реализовать базу данных и сайт-блог;
6) исследовать зависимости времени выполнения запроса выборки растений от их представления в базе данных.

1 Аналитическая часть

В аналитической части будет проведён анализ известных решений, выделены требования к приложению, формализованы данные и роли пользователей, а также выбрана база данных.

1.1 Готовые решения

Под сайтом блогом понимается сайт, на котором один или несколько пользователей способны формировать посты – записи, состоящие из текста и нескольких картинок, остальные пользователи способны читать их.

В сфере ландшафтной архитектуры содержимым постов могут выступать описания готовых решений, часто встречающихся задач, инструкции по уходу за конкретными растениями, подборки растений объединённых схожими критериями или другие темы, связанные с ландшафтной архитектурой.

Растения являются одним из основных инструментов проектирования ландшафта [2]. При этом каждой растение обладает как визуальными, так и биологическими характеристиками, которые делают невозможным использование одних и тех же растений в различных местностях. В зависимости от категории растения (например хвойные, лиственные, кустарники и др.) они обладают различными характеристиками. Таким образом для каждого проекта ландшафтному архитектору нужно подбирать подходящие растения исходя из визуальных и климатических условий проектируемого ландшафта.

Для создания базы данных сайта-блога со справочником растений были рассмотрены следующие аналоги: сайт Plantsfinder, сайт Catalog.greenmaster.by, сайт Ideas.by.

1.1.1 Сайт PlantsFinder

Сайт Plantsfinder.ru [3] – это каталог растений с автоматизированным подбором их ассортимента. На сайте представлен перечень растений по трём категориям: "лиственные деревья, кустарники и лианы "хвойные деревья и кустарники "многолетние травянистые растения". В каждой из категорий пользователь может выполнять поиск растений, задавая параметры растений, при этом искать возможно как по общим для всех категорий свойствам, таким как зона зимостойкости, отношение к свету, влажность, тип и кислотность почвы, так и по свойствам присущим конкретной категории, например период цветения у лиственных деревьев.

Также на сайте представлен раздел статей, в котором описываются готовые подборки растений, справочные материалы, решения типовых задач и другое.

На рисунке 1.1 представлен пример страницы сайта plantsfinder.

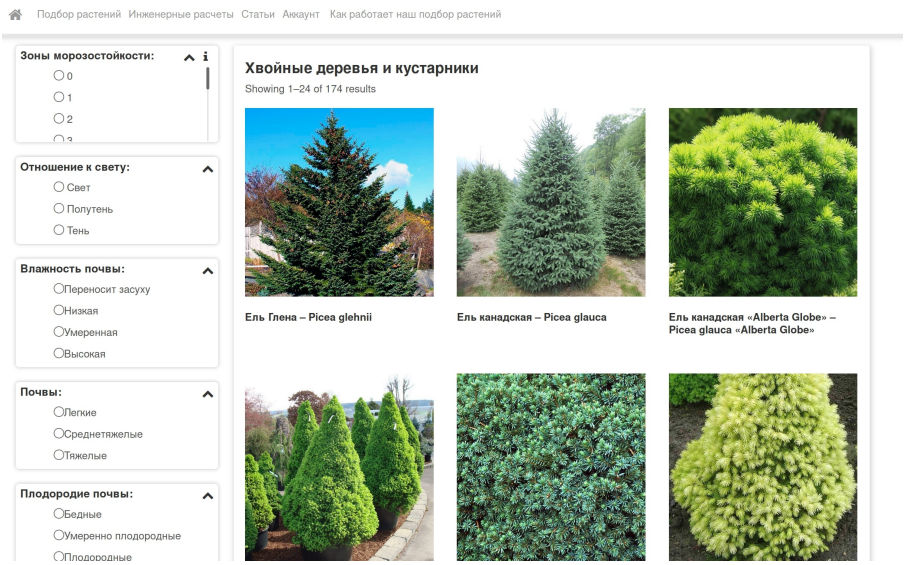


Рисунок 1.1 — Страница сайта PlantsFinder с каталогом хвойных растений

1.1.2 Сайт Catalog.greenmaster.by

Catalog.greenmaster.by [4] (далее greenmaster) – каталог растений для ландшафтных дизайнеров. Растения представленные на сайте разделены на 5 категорий: «хвойные», «лиственные деревья», «лиственные кустарники», «вересковые», «лианы». В каталоге растений возможен поиск по следующим критериям: цвет листвы, сезонность, форма кроны, устойчивость к загрязнению среды, освещение, периоды цветения и плодоношения и рекомендуемое применение. При этом для все категорий список этих параметров одинаковый. Также в виде небольших статей на сайте представлено описание каждой из доступных категорий.

На рисунке 1.2 представлена страница сайта greenmaster с каталогом хвойных растений.

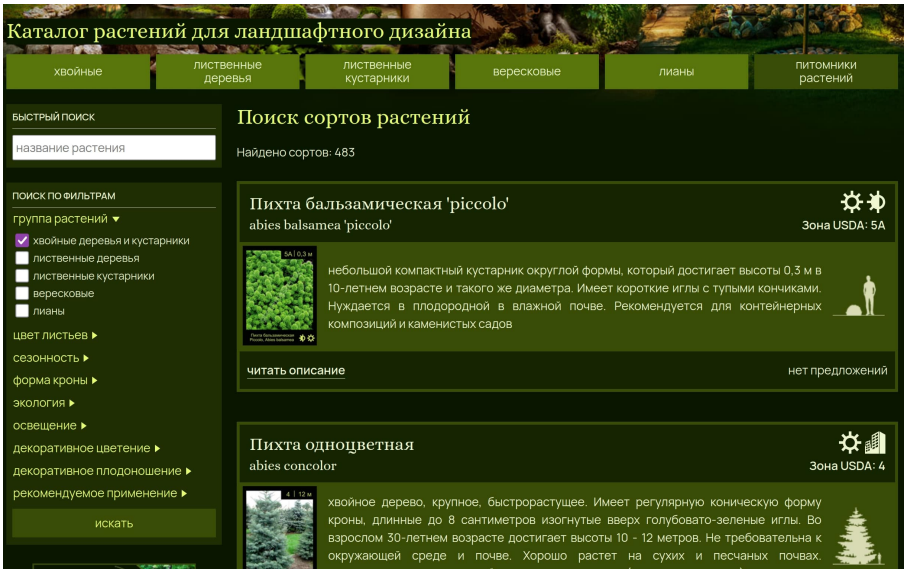


Рисунок 1.2 — Страница сайта greenmaster с каталогом хвойных растений

1.1.3 Сайт Ideas.by

Ideas.by [5] – сайт ландшафтного дизайнера Дмитрия Ломать, ведущего в нём блог. Сайт поделён на несколько страниц, на каждой из которой представлены статьи:

— портфолио – описание разработанных проектов с фотографиями и техническими деталями;

— проект – описание этапов процесса проектирования ландшафтного дизайна и важных аспектов каждого из них;

— блог – в виде постов описаны различные темы связанные с ландшафтным дизайном, в частности подборки растений, инструкции по уходу за растениями и другое.

На рисунке 1.3 представлена страница блога сайта ideas.by.

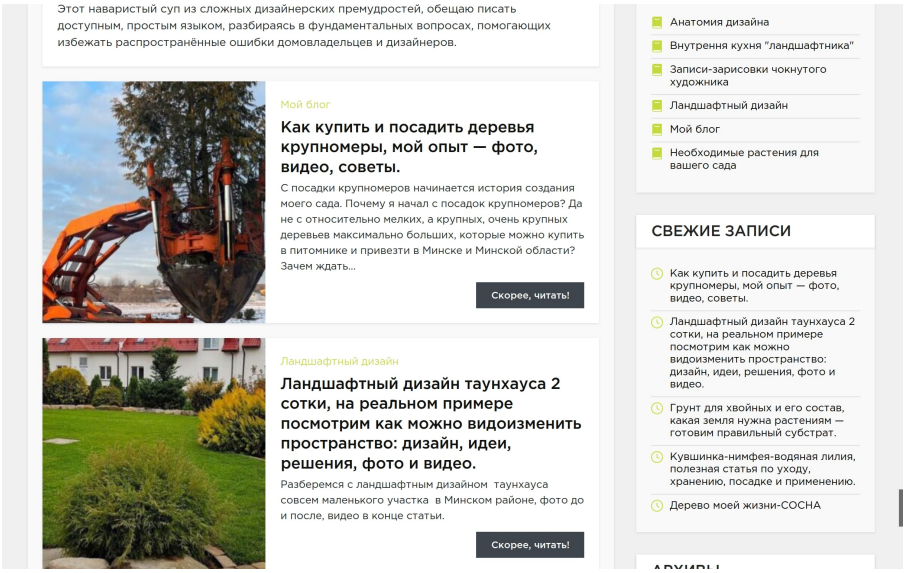


Рисунок 1.3 — Страница сайта ideas с перечнем последних статей

1.1.4 Сравнение

При анализе готовых решений были выделены следующие критерии сравнения:

1) возможность поиска растений по характеристикам;

2) наличие информации, связанной с ландшафтной архитектурой в формате постов, статей или другом;

3) связь постов или статей с растениями, описанными в них;

4) наличие готовых подборок растений;

5) возможность составления своих подборок растений;

Результаты сравнения готовых решение по сформулированным критериям представлены в таблице 1.1.

Таблица 1.1 — Сравнение известных решений

Критерий	**PlantsFinder**	**greenmaster**	**ideas.by**
Возможность поиска растений по характеристикам	+	+	-
Наличие информации, связанной с ландшафтной архитектурой в формате постов, статей или другом	+	-	+
Связь постов или статей с растения, описанными в них	-	-	-
Наличие готовых подборок растений	+	-	+
Возможность составления своих подборок растений	-	-	-

1.2 Требование к разрабатываемому приложению

Разрабатываемый сайт-блог предназначен для одного или нескольких авторов – пользователей, способных добавлять новые материалы в неё, а также обычных пользователей, которые способны читать эту информацию. При этом, так как такой функциональности не было найдено в рассмотренных известных решениях, то пользователям должна быть предоставлена возможность составления собственных подборок растений, на основе существующих в базе растений.

Таким образом, разрабатываемый сайт должен обладать следующей функциональностью:

1) просмотр общего каталога растений;

2) поиск растений по заданным свойствам;

3) просмотр постов, содержащих различную информацию, связанную с ландшафтной архитектурой;

4) поиск постов объединённых общей тематикой с помощью тегов;

5) создание пользовательских альбомов растений.

Авторы также должна быть предоставления возможность:

1) добавление, изменение и удаление растений;

2) написание и удаление постов;

1.3 Роли пользователей

В работе будут представлены следующие акторы:

— неавторизованный пользователь – пользователь, способный просматривать каталог растений и выполнять поиск растений по фильтрам. Также может просматривать посты и выполнять их поиск по содержанию или меткам;

— авторизованный пользователь – пользователь, который может делать тоже самое, что и неавторизованный, а также может создавать, удалять, редактировать свои подборки

растений;

— автор – пользователь, обладающий возможностями авторизованного пользователя, а также способный добавлять новые растения в каталог, удалять растения или изменять их свойства. Также способен писать новые посты или удалять их. Роль требует авторизации;

— администратор – пользователь, способный добавлять новые категории растений, изменять их структуру или удалять их, а также способный выдавать права автора пользователям;

Диаграмма вариантов использования разрабатываемой системы представлена на рисунке 1.4.

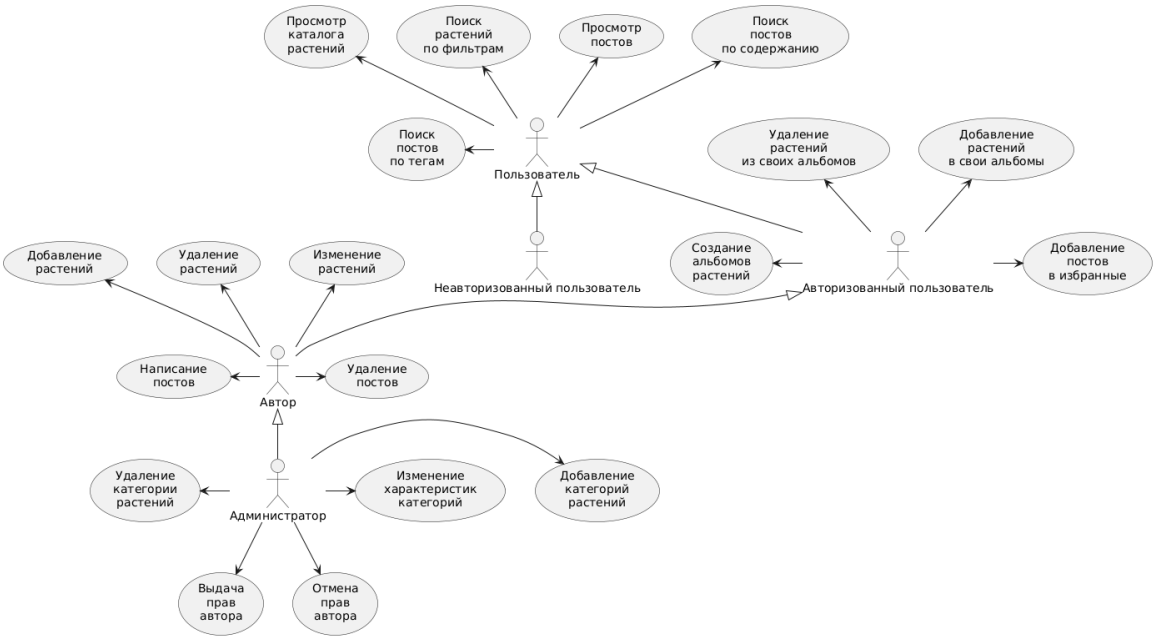


Рисунок 1.4 — Диаграмма вариантов использования разрабатываемой системы

1.4 Формализация данных

В таблице 1.2 представлены сущности, используемые в базе данных.

Таблица 1.2 — Сущности и их свойства

Сущность	**Свойство**	**Метка**	**Тип**
Пользователь	Псевдоним	Имя пользователя для авторизации	Строка
	Почта	Адрес электронной почты пользователя	Строка
	Данные аутентификации	Данные, которые могут быть использованы для аутентификации (например хеш пароля)	Последовательность байт

Таблица 1.2 — Сущности и их свойства

Сущность	**Свойство**	**Метка**	**Тип**
	Время создания	Время, в которое пользователь был создан	Временная метка
Автор	Наличие прав	Факт наличия прав автора у пользователя	Логический
	Время выдачи прав	Время, когда пользователю выдали права автора	Временная метка
	Время отмены прав	Время, когда у пользователя отняли права автора	Временная метка
Пост	Заголовок	Имя поста	Строка
	Тело	Текстовая часть поста	Строка
	Теги	Текстовые метки поста	Несколько строк
	Время создания	Время создания поста	Временная метка
	Время изменения	Время последнего изменения поста	Временная метка
	Автор	Создатель поста	Автор
	Изображения	Изображения, привязанные к посту	Изображение поста
Альбом	Название	Название альбома	Строка
	Описание	Описание альбома	Строка
	Владелец	Создатель и владелец альбома	Пользователь
	Растения	Растения, которые содержаться в альбоме	Несколько растений
	Время создания	Время, когда альбом был создан	Временная метка
	Время изменения	Время, когда альбом был изменён последний раз	Временная метка
Растение	Название	Название растения	Строка
	Латинское название	Название растения на латинском языке	Строка
	Описание	Описание растения	Строка
	Категория растения	Категория растения	Категория растения
	Фотографии растения	Фотографии с растением в кадре	Несколько фото растения
	Лицевое фото	Фотография с растением	Изображение

Таблица 1.2 — Сущности и их свойства

Сущность	**Свойство**	**Метка**	**Тип**
	Время создания	Время, когда альбом был создан	Временная метка
	Время изменения	Время, когда альбом был изменён последний раз	Временная метка
	Высота	Высота растения в метрах	Вещественное число
	Ширина	Минимальный диаметр окружности, в которую можно поместить горизонтальную проекцию растения в метрах	Вещественное число
	Отношение к свету	Требуемое освещение для растения	Строка
	Почва	Требуемый тип почвы для растения	Строка
	Влажность почвы	Требуемая влажность почвы	Строка
	Зона зимостойкости	Климатические условия, в которых растение может жить	Целое число
	Кислотность почвы	Требуемая кислотность почвы	Строка
	Период цветения	Период цветения растения	Строка
Категория растения	Название	Название категории растения	Строка
	Характеристики	Свойства, которыми могут обладать растения данной категории и их ограничения	Ассоциативный масив
	Лицевое фото	Лицевая фотография, для представления категории	Изображение
Изображение	Имя	Название изображения	Строка
	Файл	Файл изображения	Последовательность байт[1]
Изображение поста	Порядковый номер	Номер изображения в посте	Положительное число
	Изображение	Само изображение	Изображение

[1]Сам файл не будет храниться в базе данных, лишь ссылка на него.

Таблица 1.2 — Сущности и их свойства

Сущность	**Свойство**	**Метка**	**Тип**
Фото растения	Описание	Дополнительный комментарий к растению на фотографии	Строка
	Изображение	Само изображение	Изображение

На рисунке 1.5 представлена диаграмма сущность-связь в нотации Чена, на которой представлены связи сущностей разрабатываемой базы данных.

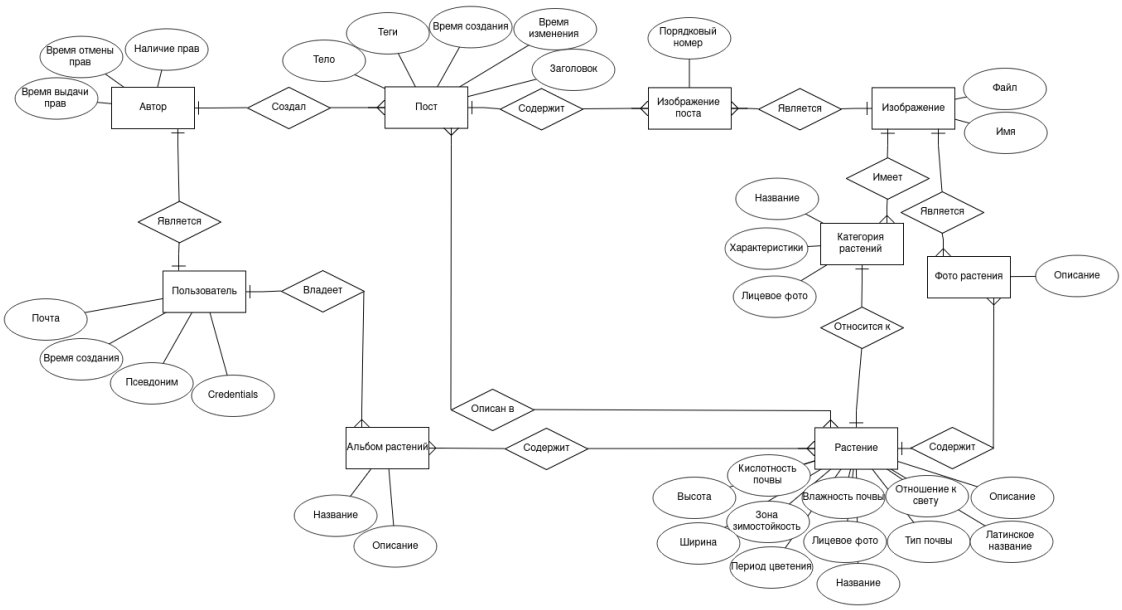


Рисунок 1.5 — Диаграмма сушность-связь в нотации Чена

1.5 Выбор базы данных

Ключевой характеристикой при выборе базы данных является модель данных – набор абстракций и приёмов, с помощью которых система пытается имитировать понятия реального мира [6].

По модели базы данных классифицируются по 3-м основным типам 1.5:

1) **дореляционные базы данных**. Такие базы данных могут иметь различные структуры и способы взаимодействия, например графовые (сетевые), иерархические базы данных (в виде дерева), базы данных на основе инвертированных списков (индексы, словари) и др. Основной чертой таких моделей данных, отличающей от остальных, является полное или почти полное отсутствие ограничений целостности и проверок корректности данных [7];

2) **реляционные базы данных** – базы данных, в которых данные хранятся в виде реляционных отношений – табличек, которые имеют фиксированную структуру, и на поля которых накладываются ограничения с помощью типов данных. Такие БД позволяют

хранить объекты с фиксированной структурой, при этом проверяя корректность самих данных.

3) **постреляционные базы данных**.Такие БД основаны на реляционных, однако отходят от их принципов в пользу более эффективного решения специфических задач [8]. Одним из примеров таких особенностей выступает возможность вложенного хранения таблиц (хранения таблиц в таблицах), что невозможно в реляционных БД.

Так как в данной работе не требуются особенности, привносимые постреляционными базами данных, однако ограничения целостности и чёткое структурирование данных играет важную роль, то для хранения рассмотренных выше сущностей была выбрана реляционная модель данных.

Вывод

В результате аналитической части был проведён анализ готовых решения для ландшафтных архитекторов, разработаны требованы для разрабатываемого ПО и формализованы данные, подлежащие хранению. Также были проанализированы модели данных и выбрана реляционная для использования в работе.

2 Конструкторская часть

В конструкторской части будет разработана реляционная схема базы данных, рассмотрены ограничения целостности, разработана ролевая модель и разработана процедура для проверки свойств у растения, присущих её категории.

2.1 Схема базы данных

В результате аналитической части была выбрана реляционная модель данных, поэтому данные будут организованы в виде связных таблиц. На рисунке 2.1 изображена реляционная схема базы данных, построенная на основе формализации данных, заданной в виде диаграммы сущность-связь 1.5.

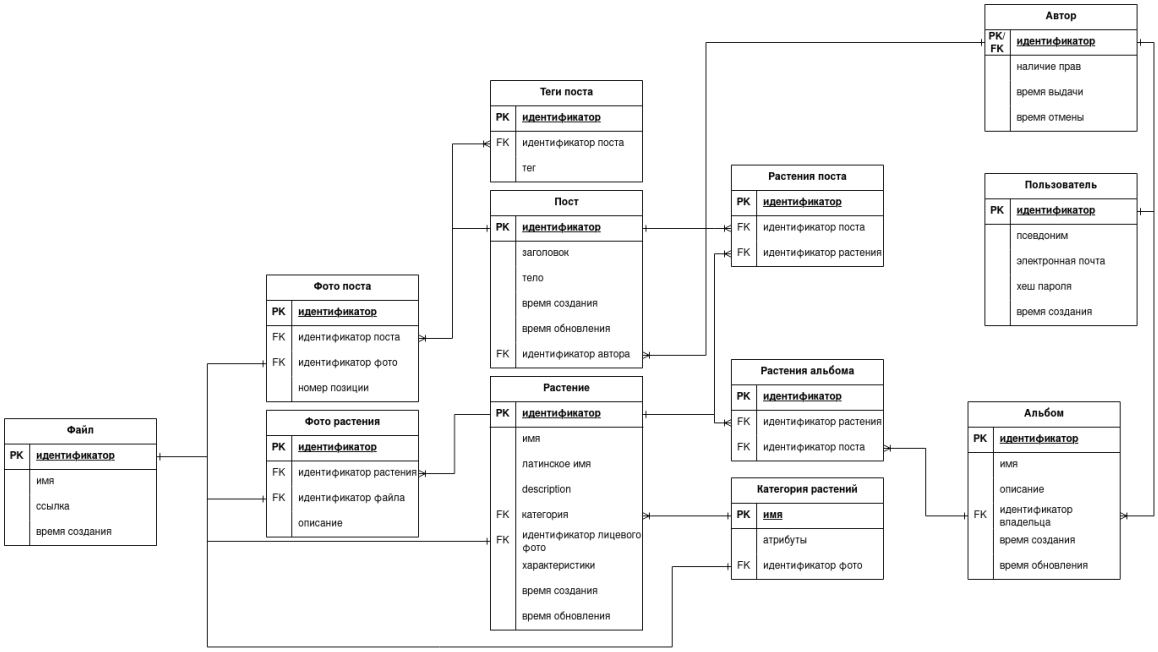


Рисунок 2.1 — Реляционная схема базы данных

Для хранения данных о характеристиках растения и об атрибутах категории растения было решено использовать ассоциативный массив, так как это позволяет хранить данные о растениях в пределах одной таблицы, несмотря на то, что характеристики растений варьируются в зависимости от категории.

Однако, так как исследовательская часть данной работы предполагает, наличие нескольких способов представления растений, то дополнительно для хранения растений была выбрана модель EAV [17], в которой растения, категория, атрибуты и значения атрибутов хранятся в виде отдельных таблицу. Данная модель, также как и хранение свойств растений в ассоциативном массиве, позволяет хранить вариантные свойства без внесения изменений в структуру базы данных [18].

2.2 Ограничения целостности

Следующие ограничения на данные введены исходя из условий целостности данных:

— все идентификаторы в базе данных должны быть уникальные;

— время добавления элементов Plant, Post и Album должно быть раньше времени обновления;

— время выдачи прав автору должно быть раньше время отмены прав, в случае если у него эти права есть, и наоборот в ином случае;

— регистрация пользователя должна быть раньше выдачи прав автора, если он является автором (время отмены прав может быть раньше создания пользователя, как показатель того, что у пользователя не забирали права);

— номера позиций фотографий в постах (place_number) должны быть уникальными в рамках одного поста, и иметь положительные значения;

— растение должно обладать только теми свойствами, которые указаны в его категории.

2.3 Триггер проверки свойств растения

Для поддержания целостности данных необходимо, чтобы растение обладало только свойствами, присущими его категории, при этом значения этих свойств должны удовлетворять ограничениям. Для этого при добавлении растения нужно проверять у него наличие и значения всех нужных свойств и отсутствие остальных. Для таких целей возможно использовать процедуру, которая выполняется при добавлении данных. Схема такой процедуры описана на рисунке 2.2

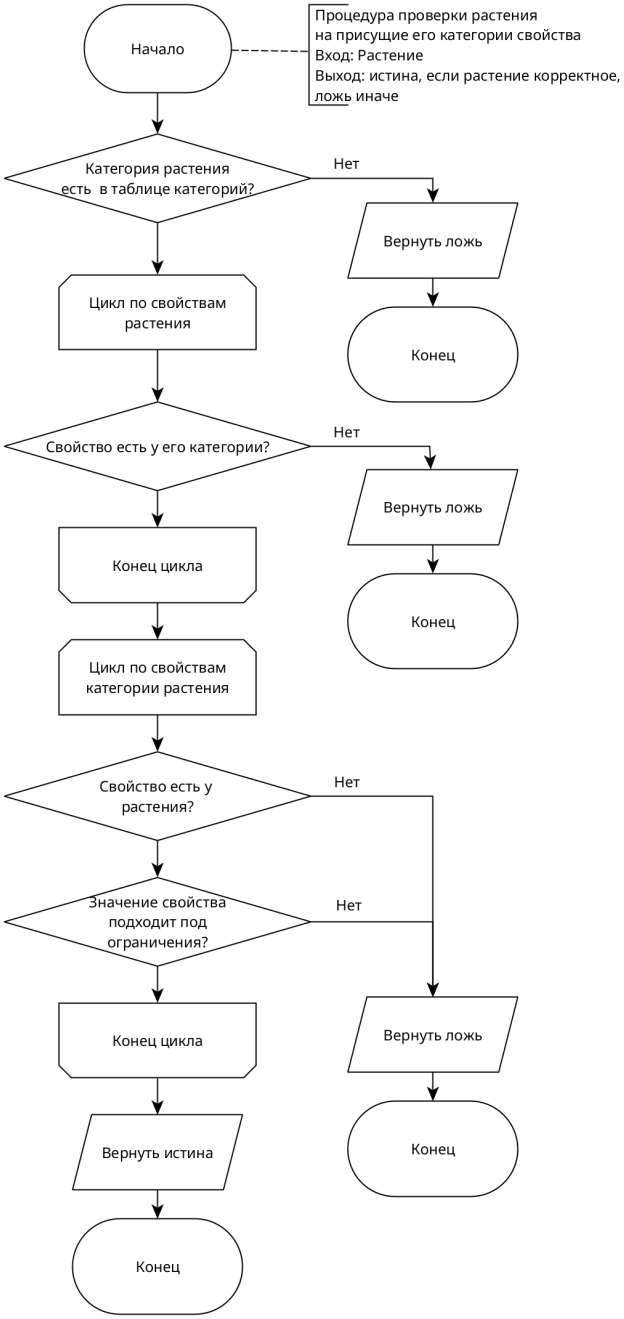


Рисунок 2.2 — Процедура проверки растения, на присущие его категории свойства

2.4 Ролевая модель

На основе пользователей приложения, описанных в аналитической части, были сформированы следующие роли и права доступа к базе данных:

— пользователь – имеет права чтения таблиц растений, постов и всех их производных им;

— авторизованный пользователь – также может добавлять, изменять и удалять записи в таблице альбомов, которые принадлежат ему;

— автор – обладает правами авторизованного пользователя, но также может добавлять, изменять и удалять записи в таблице постов и растений;

— администратор – обладает правами на добавление или удаления (отмену прав) авторов, а также на добавление новых категорий растений или изменение параметров существующих.

Описание прав доступа к таблицам на создание, чтение, изменение и удаление данных (CRUD) представлено в таблице 2.1.

Таблица 2.1 — Права доступа к таблицам

Роль	**Неавторизованный пользователь**	**Авторизованный пользователь**	**Автор**	**Администратор**
Файл	R	R	CRUD	CRUD
Фото поста	R	R	CRUD	CRUD
Фото растения	R	R	CRUD	CRUD
Теги поста	R	R	CRUD	CRUD
Пост	R	R	CRUD	CRUD
Растение	R	R	CRUD	CRUD
Растения поста	R	R	CRUD	CRUD
Растения альбома	-	CRUD[1]	CRUD[1]	CRUD
Категория	R	R	R	CRUD
Альбом	-	CRUD[1]	CRUD[1]	CRUD
Пользователь	-	-	-	CRUD
Автор	-	-	-	CRUD

Вывод

В результате конструкторского раздела была разработана реляционного схема базы данных, рассмотрены накладываемые на неё ограничения целостности и роли с правами доступа. Также была разработана процедура для проверки растения на свойства, присущие его категории.

[1] Только к тем альбомам, которыми пользователь владеет

3 Технологическая часть

В рамках технологической части будут выбраны средства реализации работы, описаны программы создания базы данных, ролей и описанного в конструкторской части триггера проверки свойств растения. Также будет описан алгоритм тестирования разработанного триггера и приложения.

3.1 Средства реализации

В качестве СУБД был выбран Postgresql [9], так как:

— средств СУБД достаточно для создания разработанной базы данных и ролевой модели;

— позволяет реализовать триггеры на добавление и обновление данных;

— СУБД Postgresql обеспечивает возможность работы с данными типа json, что может быть использована для хранения зависимых от категории данных растения.

Для хранения файлов, в частности фотографий постов и растений выбрано объектное хранилище miniO [10], так как:

— объектное хранилище обеспечивает независимость от файловой и операционной системы;

— minIO является бесплатным ПО и может быть использован на одной устройстве без кластера [11].

В качестве основного языка для разработки приложения был выбран язык Go [12], так как:

— средств языка достаточно для реализации разрабатываемого приложения;

— в языке есть готовые библиотеки, позволяющие реализовать взаимодействие с Postgresql, например [13].

— язык имеет официальную библиотеку для взаимодействия с minIO [14].

3.2 Создание базы данных

Так как Postgresql предоставляет возможность хранения данных в полях типа JSON, то характеристики растений, которые зависят от категории, хранятся именно в поле типа JSON. Это позволяет для растений каждой категории хранить только те параметры, которые у неё есть. При этом у самой категории хранится также есть поле типа JSON, в котором описан формат и ограничения характеристик растений.

На основе разработанной схемы и описания данных и конструкторской части были выделены 3 типа полей характеристик растений, которые описаны в таблице 3.1

Таблица 3.1 — Типы характеристик растения

Тип	**Обозначение в БД**	**Ограничения**	**Характеристики**
Символьный литерал	string	**options** – массив вариантов значения характеристики. Если **options** не указан, значит характеристика может принимать любое значение.	Отношение к свету, период цветения, тип почвы, влажность почвы.
Целое число	number	**min** – минимальное значение, которое может принимать характеристика. **max** – максимальное значение, которое может принимать характеристика.	Кислотность почвы, зона зимостойкости.
Вещественное число	float	**min** – минимальное значение, которое может принимать характеристика. **max** – максимальное значение, которое может принимать характеристика.	Высота, диаметр

Таким образом, каждая характеристика растения в таблице категорий описывается именем и типом из таблицы 3.1.

Создание базы данных происходит в 2 этапа. На первом создаются таблицы и ограничения к ним, а на втором заполняется таблица категорий, с указанием характеристик, присущей каждой из них. Этапы представлены в приложении А в листингах 4.1 и 4.2 соответственно.

3.3 Роли и их права

В конструкторской части были выделены 4 роли:

— пользователь;

— авторизованный пользователь;

— автор;

— администратор;

Для каждой из этих ролей была создана отдельная роль в postgresql, которая определяет какие операции над таблицами эта роль может производить. Для ограничения, на изменение и удаление чужих постов и альбомов, а также добавления их не под своим идентификатором, были использованы политики защиты строк [15], которые ограничивают доступ пользователям к альбомам, которые были созданы не ими и к изменению постов чужих авторов. Код создания этих ролей представлен в приложении А, листингах 4.3 - 4.6.

3.4 Триггер проверки свойств растений

Для проверки правильности добавляемых в базу данных растений был разработан триггер, которые на каждое добавление или изменений растений проверяет новые записи. Если одно из добавляемых или изменяемых растений вызывает исключение в триггере, то вся операция не выполняется с ошибкой.

Код разработанного триггера представлен в приложении А, в листинге 4.7.

3.5 Сайт-блог

В качестве пользовательского интерфейса был разработан сайт-блог со справочником растений, предоставляющий описанную в аналитической части функциональность. Разработанное ПО разделено на компоненты, описание которых представлено в UML формате на рисунке 3.1

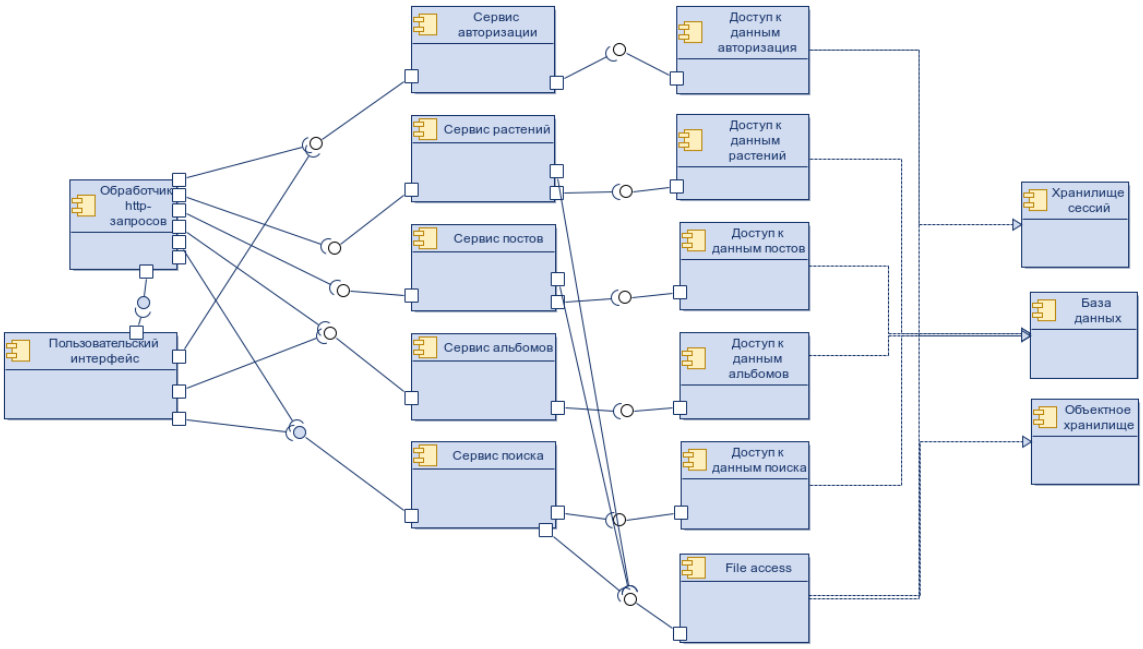


Рисунок 3.1 — UML-диаграмма компонентов разработанного ПО

3.6 Тестирование

3.6.1 Тестирование приложения

Модульное и интеграционное тестирование приложения проводилось с помощью стандартного пакета Go – testing, и стороннего пакета testify.

В качестве меры качества покрытия кода использовалась метрика стандартной утилиты cover входящей в состав Go, которая измеряет покрытие кода как процент от выполненных хотя бы единожды выражений [16].

Тестами были покрыты модули логики приложения, взаимодействия с базой данных, объектным хранилищем.

Покрытие кода составило **32.3%**.

3.6.2 Тестирование триггера проверки характеристик растений

Тестирование триггера проводилось с помощью программ на языке GO. Для тестов триггера создавалась отдельная категория, с полями каждого вида. Поля тестовой категории представлены в таблице 3.2.

Таблица 3.2 — Поля тестовой категории растений

Поле	**Тип**	**Ограничения**
int_field	number	min:1 max: 10
float_field	float	min:0.5
select_field	string	options: ["opt1 "opt2"]
string_field	string	

Были выделены следующие классы эквивалентности:

— корректные характеристики растения
— некорректная категория
— отсутствует необходимое поле "select_field"
— отсутствует необходимое поле "int_field"
— лишнее поле
— некорректное значение для поля типа number
— вещественное число в поле типа number
— значение в поле типа number ниже минимального ограничения
— значение в поле типа number выше максимального ограничения
— некорректное значение для поля типа float
— значение в поле типа float ниже минимального ограничения
— целое значение в поле типа float
— некорректное значение для поля типа string
— значение вне допустимых для поля типа string с ограничениями

Каждым из классов эквивалентности был протестирован на триггер на операциях вставки и обновления. Общий алгоритм тестирования триггера имел следующий вид:

1) Создание тестовой категории в базе данных и триггера на добавление и обновление;
2) Загрузка тестового изображения для создания растений;
3) Цикл по всем тестам:

3.1) создание растения с характеристиками теста в базе данных;

3.2) если тест позитивный:

- проверка, что триггер не вызвал ошибку;
- проверка, что растение успешно добавлено в таблицу растений;

3.3) иначе:

- проверка, что триггер вызвал ошибку;
- проверка, что растение не добавлен в базу данных;

3.4) очистка таблицы растений;

4) Цикл по всем тестам:

4.1) создать корректное растение;

4.2) изменить созданное растение, согласно тестовым данным;

4.3) если тест позитивный:

- проверка, что триггер не вызвал ошибку;
- проверка, что растение успешно изменено;

4.4) иначе:

- проверка, что триггер вызвал ошибку;
- проверка, что растение не изменено;

Вывод

В результате технологической части были выбраны средства реализации работы, созданы база данных, роли, триггер проверки свойств растений. Также было разработано и протестировано приложение сайта-блога со справочником растений.

4 Исследовательская часть

В данном разделе будет произведена постановка исследования, описана методология проведения замеров, приведены и проанализированы их результаты.

4.1 Цель исследования

Объектом данного исследования является зависимость времени получения данных запросов к базе данных растений от её схемы.

На рисунке 4.1 представлена формализованная постановка исследования запроса на получение данных.



Рисунок 4.1 — Диаграмма верхнего уровня в нотации idef0 для запроса получения данных

Из рисунка 4.1 следует, что параметрами в исследовании выступают:

— объём баз данных;

— параметры запросы (дополнительные условия, фильтры)

4.2 Описание исследования

Для исследования база данных упрощена, и содержит только информацию напрямую связанную с растениями. В качестве схем выбраны две модели:

— JSON – характеристики растений хранятся в JSON, также, как описано в технологической части;

— Entity-Attribute-Value (EAV) – растения, категории, атрибуты и значения атрибутов хранятся отдельными таблицами [17].

Создание баз данных для СУБД Postgresql приведены в приложении А, в листингах 4.8 и 4.9.

Параметрами запроса на выборку данных являются ограничения и условия на выбираемые записи. Выделенные типы запросов с различными параметрами представлены в таблице 4.1.

Таблица 4.1 — Типы запросов

Тип	**Описание**
all	Получение всех растений, без дополнительных условий и ограничений.
height_m_filter	Получение растений с одним числовым фильтром (условием) на параметр высоты растения.
height_m_and_diameter_m_filter	Получение растений с двумя числовыми фильтрами (условиями) на параметры высоты и диаметра растения.
string_description_filter	Получение растений с одним фильтром (условием) на вхождение заданной подстроки в описание (строковой литерал) растения.
lot_of_filters	Получение растения с 5-ю фильтрами различных типов (числовые, вхождение подстроки, равенство строковых литералов).
category_filter	Получение растений заданной категории, то есть с фильтром на равенство строковых литералов.

Общий алгоритм исследования имеет вид:

Входные данные:

— типы запросов;

— sizes – объёмы выборок данных.

Выходные данные:

— время выполнения запроса для каждого типа запроса и объёма выборки.

Алгоритм

1) Создать базы данных JSON и EAV;
2) Для каждого размера size из sizes:
    2.1) заполнить обе базы данных случайными растениями объёмом size (обе базы данных заполняются одними и теми же растениями)
    2.2) для каждого типа запроса:
        2.2.1) Цикл от 1 по 10:
            — выполнить запрос с префиксом «Explain analyze» [19] к базе JSON;
            — выполнить запрос с префиксом «Explain analyze» [19] к базе EAV;
            — вытащить из результатов времена выполнения запросов;
            — записать времена выполнения запросов;
        2.2.2) усреднить времена выполнения запросов по каждой из баз данных;

2.3) очистить базы данных;

4.3 Результаты исследования

Замеры проводились на устройстве со следующими характеристиками:

— Процессор: AMD Ryzen 7 5800H (16 ядер, 4.46 ГГц);

— Оперативная память: 16 ГБ;

— Операционная система: CachyOS x86_64 (ядро linux 6.14.4-2);

Результаты исследования приведены в таблице 4.2 в приложении А.

На рисунке 4.2 изображены графики зависимости времени выполнения от объёма выборки по разным типам запроса.

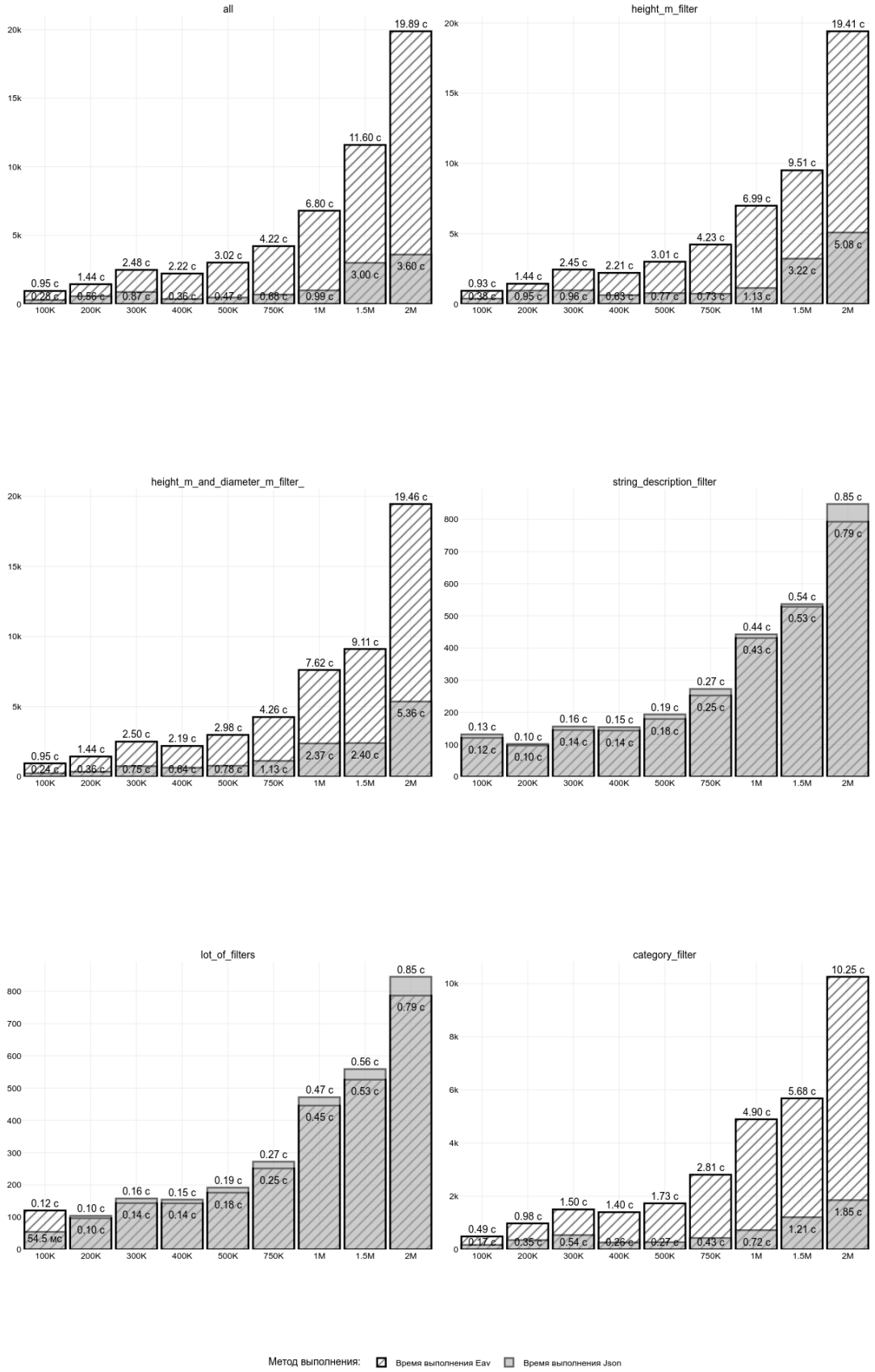


Рисунок 4.2 — Графики зависимости времени выполнения от объёма выборки по разным типам запроса

Вывод

В результате исследования было выявлено, что:

— Для большинства типов запросов (all, height_m_filter, height_m_and_diameter_m_filter, category_filter) обработка происходит быстрее в базе данных, которая хранит характеристики в формате JSON (3-5 раз быстрее);

— Запросы с обработкой строк и большим число фильтров обрабатывались медленнее в базе данных, которая хранит характеристики в формате JSON (10%)

ЗАКЛЮЧЕНИЕ

В результате курсовой работы был проведён анализ предметной области ландшафтного дизайна, рассмотрены готовые решения для справочников растений, блогов ландшафтных дизайнеров. Были формализованы данные о растениях, пользователях, постах, выделены роли пользователей и их сценарии использования.

Была спроектирована реляционная база данных, для хранения данных о растениях, пользователях и постах сайта-блога, описаны ограничения целостности, сформулированы ограничения доступа и действий каждой роли в системе.

В результате работы была разработан сайт-блог ландшафтного архитектора с базой данных растений. Разработанное ПО позволяет выполнять поиск растений и постов по заданным критериям, создавать коллекции растений, добавлять новые растения и посты.

В ходе исследования была изучена зависимость времени выполнения запроса получения данных в зависимости от схемы хранения характеристик растений. По результатом исследования стало, что хранение характеристик растений в виде JSON позволяет ускорить время запроса в 3-5 раз по сравнению с моделью EAV.

В ходе работы поставленные цель и задачи были выполнены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Неровня И. О. ИСПОЛЬЗОВАНИЕ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ В ЛАНДШАФТНОМ ПРОЕКТИРОВАНИИ // novainfo. - 2017. - С. 38-46.

2. Теория ландшафтной архитектуры и методология проектирования : краткий курс лекций. - Саратов: ФГБОУ ВО «Саратовский ГАУ», 2016. - 73 с.

3. Plantsfinder.ru URL: https://Plantsfinder.ru (дата обращения: 11.03.2025).

4. Каталог растений для ландшафтного дизайна URL: https://catalog.greenmaster.by/ (дата обращения: 11.03.2025).

5. Блог Дмитрия Ломать URL: https://ideas.by/blog/ (дата обращения: 11.03.2025).

6. Комаров В. И. Путеводитель по базам данных. - М.: ДМК-Пресс, 2024. - 520 с.

7. Конспекты лекций курса «Базы данных» за 2024 год. МГТУ им. Н.Э. Баумана, кафедра «Программное обеспечение ЭВМ и информационные технологии»

8. Парфенов, Ю. П. Постреляционные хранилища данных : учеб. пособие. - Екатеринбург: изд-во Урал, 2016. - 120 с.

9. PostgreSQL: The World's Most Advanced Open Source Relational Database // URL: https://www.postgresql.org/ (дата обращения: 02.05.2025).

10. MinIO // URL: https://min.io/ (дата обращения: 02.05.2025).

11. MinIO Object Storage for Linux // minIO URL: https://min.io/docs/minio/linux/index.html (дата обращения: 15.05.2025).

12. The Go programming language // URL: https://go.dev/ (дата обращения: 02.05.2025).

13. pgx - PostgreSQL Driver and Toolkit // Github URL: https://github.com/jackc/pgx (дата обращения: 02.05.2025).

14. Go Quickstart Guide // MinIO URL: https://min.io/docs/minio/linux/developers/go/minio-go.html (дата обращения: 02.05.2025).

15. Row security policies // Postgresql URL: https://www.postgresql.org/docs/current/ddl-rowsecurity.html (дата обращения: 03.05.2025).

16. The cover story // The Go Programming Lamguage URL: https://go.dev/blog/cover (дата обращения: 03.05.2025).

17. Shivani Batra, Shelly Sachdeva, and Subhash Bhalla Entity Attribute Value Style Modeling Approach for Archetype Based Data // Information. - 2018. - №9

18. Rakesh Agrawal, Amit Somani, Yirong Xu Storage and Querying of E-Commerce Data // IBM Almaden Research Center. - 2001.

19. Explain // PostgreSQL: The World's Most Advanced Open Source Relational Database URL: https://www.postgresql.org/docs/current/sql-explain.html (дата обращения: 01.05.2025).

Приложение А

Листинг 4.1 — Создание базы данных и ограничений целостности

```
CREATE TABLE IF NOT EXISTS "file" (
id UUID PRIMARY KEY,
name TEXT NOT NULL,
url TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS app_user (
id UUID PRIMARY KEY,
username TEXT NOT NULL UNIQUE,
email TEXT NOT NULL UNIQUE,
password_hash TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS author (
id UUID PRIMARY KEY,
has_rights BOOLEAN NOT NULL,
grant_at TIMESTAMPTZ DEFAULT NOW(),
revoke_at TIMESTAMPTZ DEFAULT NOW(),
FOREIGN KEY (id) REFERENCES app_user(id),
CONSTRAINT rights_timing_check CHECK (
(has_rights AND grant_at > revoke_at) OR
(NOT has_rights AND revoke_at > grant_at)
)
);

CREATE TABLE IF NOT EXISTS plant_category (
name TEXT PRIMARY KEY,
attributes JSONB NOT NULL,
photo_id UUID,
CONSTRAINT photo_id_fk FOREIGN KEY (photo_id) REFERENCES "file"(id)
);

CREATE TABLE IF NOT EXISTS plant (
id UUID PRIMARY KEY,
name TEXT NOT NULL,
latin_name TEXT NOT NULL,
description TEXT NOT NULL,
```

```
category TEXT,
main_photo_id UUID,
specification JSONB NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
CONSTRAINT main_photo_id_fk FOREIGN KEY (main_photo_id) REFERENCES "file"(
    id),
CONSTRAINT category_fk FOREIGN KEY (category) REFERENCES plant_category(
    name),
CONSTRAINT created_before_updated CHECK (created_at <= updated_at)
);

CREATE TABLE IF NOT EXISTS plant_photo (
id UUID PRIMARY KEY,
plant_id UUID NOT NULL,
file_id UUID NOT NULL,
description TEXT NOT NULL,
FOREIGN KEY (plant_id) REFERENCES plant(id),
FOREIGN KEY (file_id) REFERENCES "file"(id)
);

CREATE TABLE IF NOT EXISTS post (
id UUID PRIMARY KEY,
title TEXT NOT NULL,
body TEXT NOT NULL,
author_id UUID,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
FOREIGN KEY (author_id) REFERENCES author(id),
CONSTRAINT created_before_updated CHECK (created_at <= updated_at)
);

CREATE TABLE IF NOT EXISTS post_photo (
id UUID PRIMARY KEY,
post_id UUID NOT NULL,
file_id UUID NOT NULL,
place_number int NOT NULL,
FOREIGN KEY (post_id) REFERENCES post(id),
FOREIGN KEY (file_id) REFERENCES "file"(id),
CONSTRAINT place_number_positive CHECK (place_number > 0),
CONSTRAINT place_number_unique UNIQUE (post_id, place_number)
```

```
);

CREATE TABLE IF NOT EXISTS plant_post (
id UUID PRIMARY KEY,
plant_id UUID NOT NULL,
post_id UUID NOT NULL,
FOREIGN KEY (plant_id) REFERENCES plant(id),
FOREIGN KEY (post_id) REFERENCES post(id)
);

CREATE TABLE IF NOT EXISTS post_tag (
id UUID PRIMARY KEY,
post_id UUID NOT NULL,
tag TEXT NOT NULL,
FOREIGN KEY (post_id) REFERENCES post(id)
);

CREATE TABLE IF NOT EXISTS album (
id UUID PRIMARY KEY,
name TEXT NOT NULL,
description TEXT NOT NULL,
owner_id UUID NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
FOREIGN KEY (owner_id) REFERENCES app_user(id),
CONSTRAINT created_before_updated CHECK (created_at <= updated_at)
);

CREATE TABLE IF NOT EXISTS plant_album (
id UUID PRIMARY KEY,
plant_id UUID NOT NULL,
album_id UUID NOT NULL,
FOREIGN KEY (plant_id) REFERENCES plant(id),
FOREIGN KEY (album_id) REFERENCES album(id)
);

-- check that user is created before grant_at in author
CREATE OR REPLACE FUNCTION check_user_created_before_grant()
RETURNS TRIGGER AS $$
BEGIN
IF EXISTS (
```

```
SELECT 1 FROM app_user
WHERE id = NEW.id AND created_at >= NEW.grant_at
) THEN
RAISE EXCEPTION 'User must be created before grant_at';
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER author_created_before_grant
BEFORE INSERT OR UPDATE ON author
FOR EACH ROW EXECUTE FUNCTION check_user_created_before_grant();
```

Листинг 4.2 — Создание категорий растений в базе данных

```
INSERT INTO plant_category (name, attributes, photo_id) VALUES
('coniferous',
'{
  "height_m": {
    "type": "float",
    "min": 0.0
  },
  "diameter_m": {
    "type": "float",
    "min": 0.0
  },
  "soil_acidity": {
    "type": "number",
    "min": 0.0
  },
  "soil_moisture": {
    "type": "select",
    "options": [
    "dry",
    "low",
    "medium",
    "high"
    ]
  },
  "light_relation": {
    "type": "select",
    "options": [
    "light",
```

```
    "halfshadow",
    "shadow"
    ]
  },
  "soil_type": {
    "type": "select",
    "options": [
    "light",
    "medium",
    "heavy"
    ]
  },
  "winter_hardiness": {
    "type": "number",
    "min": 1.0,
    "max": 11.0
  }
}',
NULL),
('deciduous',
'{
  "height_m": {
    "type": "float",
    "min": 0.0
  },
  "diameter_m": {
    "type": "float",
    "min": 0.0
  },
  "soil_acidity": {
    "type": "number",
    "min": 0.0
  },
  "soil_moisture": {
    "type": "select",
    "options": [
    "dry",
    "low",
    "medium",
    "high"
    ]
```

```
  },
  "light_relation": {
    "type": "select",
    "options": [
    "light",
    "halfshadow",
    "shadow"
    ]
  },
  "soil_type": {
    "type": "select",
    "options": [
    "light",
    "medium",
    "heavy"
    ]
  },
  "winter_hardiness": {
    "type": "number",
    "min": 1.0,
    "max": 11.0
  },
  "flowering_period": {
    "type": "select",
    "options": [
    "spring",
    "summer",
    "autumn",
    "winter",
    "january",
    "february",
    "march",
    "april",
    "may",
    "june",
    "july",
    "august",
    "september",
    "october",
    "november",
    "december"
```

```
    ]
  }
}', NULL);
```

Листинг 4.3 — Создание роли пользователя

```
CREATE ROLE noauth_user;
GRANT SELECT ON TABLE plant TO noauth_user;
GRANT SELECT ON TABLE post TO noauth_user;
GRANT SELECT ON TABLE plant_category TO noauth_user;
GRANT SELECT ON TABLE plant_photo TO noauth_user;
GRANT SELECT ON TABLE post_photo TO noauth_user;
GRANT SELECT ON TABLE plant_post TO noauth_user;
GRANT SELECT ON TABLE post_tag TO noauth_user;
GRANT SELECT ON TABLE "file" TO noauth_user;
```

Листинг 4.4 — Создание роли авторизованного пользователя

```
CREATE ROLE auth_user;
GRANT noauth_user TO auth_user;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE album TO auth_user;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE plant_album TO auth_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO auth_user;

ALTER TABLE album ENABLE ROW LEVEL SECURITY;
CREATE POLICY album_owner_policy ON album
FOR ALL
TO auth_user
USING (owner_id = current_user_id())
WITH CHECK (owner_id = current_user_id());

ALTER TABLE plant_album ENABLE ROW LEVEL SECURITY;
CREATE POLICY plant_album_owner_policy ON plant_album
FOR ALL
TO auth_user
USING (album_id IN (SELECT id from album WHERE owner_id = current_user_id
    ()))
WITH CHECK (album_id IN (SELECT id from album WHERE owner_id =
    current_user_id()));
```

Листинг 4.5 — Создание роли автора

```
CREATE ROLE author;
GRANT auth_user TO author;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE post TO author;
```

```
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE post_photo TO author;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE plant_post TO author;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE post_tag TO author;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE plant TO author;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE plant_photo TO author;
GRANT INSERT, SELECT, UPDATE, DELETE ON TABLE "file" TO author;

ALTER TABLE plant ENABLE ROW LEVEL SECURITY;
CREATE POLICY plant_select_policy ON plant
FOR SELECT
TO PUBLIC
USING (TRUE);

CREATE POLICY plant_insert_policy ON plant
FOR INSERT
TO author
WITH CHECK ((SELECT current_author_id()) IS NOT NULL);

CREATE POLICY plant_update_policy ON plant
FOR UPDATE
TO author
USING ((SELECT current_author_id()) IS NOT NULL)
WITH CHECK ((SELECT current_author_id()) IS NOT NULL);

CREATE POLICY plant_delete_policy ON plant
FOR DELETE
TO author
USING ((SELECT current_author_id()) IS NOT NULL);

ALTER TABLE post ENABLE ROW LEVEL SECURITY;
CREATE POLICY post_select_policy ON post
FOR SELECT
TO PUBLIC
USING (TRUE);

CREATE POLICY post_insert_policy ON post
FOR INSERT
TO author
WITH CHECK (author_id = current_author_id());

CREATE POLICY post_update_policy ON post
```

```
FOR UPDATE
TO author
USING (author_id = current_author_id())
WITH CHECK (author_id = current_author_id());

CREATE POLICY post_delete_policy ON post
FOR DELETE
TO author
USING (author_id = current_author_id());

ALTER TABLE post_photo ENABLE ROW LEVEL SECURITY;
CREATE POLICY post_photo_select_policy ON post_photo
FOR SELECT
TO PUBLIC
USING (TRUE);

CREATE POLICY post_photo_insert_policy ON post_photo
FOR INSERT
TO author
WITH CHECK (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));

CREATE POLICY post_photo_update_policy ON post_photo
FOR UPDATE
TO author
USING (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()))
WITH CHECK (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));

CREATE POLICY post_photo_delete_policy ON post_photo
FOR DELETE
TO author
USING (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));



ALTER TABLE post_tag ENABLE ROW LEVEL SECURITY;
CREATE POLICY post_tag_select_policy ON post_tag
FOR SELECT
TO PUBLIC
```

```
USING (TRUE);

CREATE POLICY post_tag_insert_policy ON post_tag
FOR INSERT
TO author
WITH CHECK (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));

CREATE POLICY post_tag_update_policy ON post_tag
FOR UPDATE
TO author
USING (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()))
WITH CHECK (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));

CREATE POLICY post_tag_delete_policy ON post_tag
FOR DELETE
TO author
USING (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));

ALTER TABLE plant_post ENABLE ROW LEVEL SECURITY;

CREATE POLICY plant_post_select_policy ON plant_post
FOR SELECT
TO PUBLIC
USING (TRUE);

CREATE POLICY plant_post_insert_policy ON plant_post
FOR INSERT
TO author
WITH CHECK (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));

CREATE POLICY plant_post_update_policy ON plant_post
FOR UPDATE
TO author
USING (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()))
WITH CHECK (post_id in (SELECT id FROM post WHERE post.author_id =
```

```
    current_author_id()));

CREATE POLICY plant_post_delete_policy ON plant_post
FOR DELETE
TO author
USING (post_id in (SELECT id FROM post WHERE post.author_id =
    current_author_id()));


ALTER TABLE plant_photo ENABLE ROW LEVEL SECURITY;
CREATE POLICY plant_photo_select_policy ON plant_photo
FOR SELECT
TO PUBLIC
USING (TRUE);

CREATE POLICY plant_photo_insert_policy ON plant_photo
FOR INSERT
TO author
WITH CHECK ((SELECT current_author_id()) IS NOT NULL);

CREATE POLICY plant_photo_update_policy ON plant_photo
FOR UPDATE
TO author
USING ((SELECT current_author_id()) IS NOT NULL)
WITH CHECK ((SELECT current_author_id()) IS NOT NULL);

CREATE POLICY plant_photo_delete_policy ON plant_photo
FOR DELETE
TO author
USING ((SELECT current_author_id()) IS NOT NULL);
```

Листинг 4.6 — Создание роли администратора

```
CREATE ROLE admin;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin;
```

Листинг 4.7 — Создание триггера проверки характеристик растения

```
CREATE OR REPLACE FUNCTION validate_plant_specification()
RETURNS TRIGGER AS $$
DECLARE
category_attrs JSONB;
attr_name TEXT;
```

```
attr_def JSONB;
attr_value JSONB;
attr_type TEXT;
min_val NUMERIC;
max_val NUMERIC;
options TEXT[];
option TEXT;
valid_option BOOLEAN;
num_value NUMERIC;
spec_attr TEXT;
allowed_attrs TEXT[];
string_value TEXT;
BEGIN
SELECT attributes INTO category_attrs
FROM plant_category
WHERE name = NEW.category;

IF category_attrs IS NULL THEN
RAISE EXCEPTION 'Plant category "%" does not exist', NEW.category;
END IF;

SELECT array_agg(key) INTO allowed_attrs
FROM jsonb_object_keys(category_attrs) AS key;

FOR spec_attr IN SELECT jsonb_object_keys(NEW.specification)
LOOP
IF NOT spec_attr = ANY(allowed_attrs) THEN
RAISE EXCEPTION 'Specification contains extra attribute "%" not defined in
    category "%"',
spec_attr, NEW.category;
END IF;
END LOOP;

FOR attr_name IN SELECT jsonb_object_keys(category_attrs)
LOOP
attr_def := category_attrs->attr_name;
attr_value := NEW.specification->attr_name;

IF attr_value IS NULL THEN
RAISE EXCEPTION 'Missing required attribute "%" in specification',
   attr_name;
```

```
END IF;

attr_type := attr_def->>'type';

-- Validate based on type
CASE attr_type
WHEN 'float', 'number' THEN
-- Check if value is numeric
IF jsonb_typeof(attr_value) != 'number' THEN
RAISE EXCEPTION 'Attribute "%" must be a number, got %',
attr_name, jsonb_typeof(attr_value);
END IF;

BEGIN
num_value := (attr_value::TEXT)::NUMERIC;
EXCEPTION WHEN OTHERS THEN
RAISE EXCEPTION 'Attribute "%" must be a valid number', attr_name;
END;

IF attr_type = 'number' AND num_value % 1 != 0 THEN
RAISE EXCEPTION 'Attribute "%" must be an integer, got %',
attr_name, attr_value;
END IF;

IF attr_def ? 'min' THEN
min_val := (attr_def->>'min')::NUMERIC;
IF num_value < min_val THEN
RAISE EXCEPTION 'Attribute "%" must be at least %, got %',
attr_name, min_val, attr_value;
END IF;
END IF;

IF attr_def ? 'max' THEN
max_val := (attr_def->>'max')::NUMERIC;
IF num_value > max_val THEN
RAISE EXCEPTION 'Attribute "%" must be at most %, got %',
attr_name, max_val, attr_value;
END IF;
END IF;

WHEN 'string' THEN
```

```
IF jsonb_typeof(attr_value) != 'string' THEN
RAISE EXCEPTION 'Attribute "%" must be a string, got %',
attr_name, jsonb_typeof(attr_value);
END IF;

string_value := attr_value #>> '{}';

IF attr_def ? 'options' THEN
options := ARRAY(SELECT jsonb_array_elements_text(attr_def->'options'));
valid_option := FALSE;

FOREACH option IN ARRAY options LOOP
IF option = string_value THEN
valid_option := TRUE;
EXIT;
END IF;
END LOOP;

IF NOT valid_option THEN
RAISE EXCEPTION 'Attribute "%" value "%" is not in allowed options: %',
attr_name, string_value, options;
END IF;
END IF;

ELSE
RAISE EXCEPTION 'Unknown attribute type "%" for attribute "%"',
attr_type, attr_name;
END CASE;
END LOOP;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS plant_specification_validation ON plant;
CREATE TRIGGER plant_specification_validation
BEFORE INSERT OR UPDATE ON plant
FOR EACH ROW EXECUTE FUNCTION validate_plant_specification();
```

Листинг 4.8 — Создание базы данных растений с хранением характеристик в JSON.

```
CREATE TABLE IF NOT EXISTS plant_category (
name TEXT PRIMARY KEY,
```

```
attributes JSONB NOT NULL,
photo_id UUID
);

CREATE TABLE IF NOT EXISTS "file" (
id UUID PRIMARY KEY,
name TEXT NOT NULL,
url TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS plant (
id UUID PRIMARY KEY,
name TEXT NOT NULL,
latin_name TEXT NOT NULL,
description TEXT NOT NULL,
category TEXT,
main_photo_id UUID,
specification JSONB NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
FOREIGN KEY (main_photo_id) REFERENCES file(id),
FOREIGN KEY (category) REFERENCES plant_category(name)
);

CREATE TABLE IF NOT EXISTS plant_photo (
id UUID PRIMARY KEY,
plant_id UUID NOT NULL,
file_id UUID NOT NULL,
description TEXT NOT NULL,
FOREIGN KEY (plant_id) REFERENCES plant(id),
FOREIGN KEY (file_id) REFERENCES "file"(id)
);
```

Листинг 4.9 — Создание базы данных растений с хранением характеристик по модели EAV.

```
CREATE TABLE IF NOT EXISTS "file" (
id UUID PRIMARY KEY,
name TEXT NOT NULL,
url TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE IF NOT EXISTS plant_category (
name TEXT PRIMARY KEY,
photo_id UUID,
FOREIGN KEY (photo_id) REFERENCES file(id)
);

CREATE TABLE IF NOT EXISTS attribute (
id UUID PRIMARY KEY,
name TEXT NOT NULL UNIQUE,
data_type TEXT NOT NULL CHECK (data_type IN ('float', 'number', 'select',
    'text')),
min_value FLOAT,
max_value FLOAT
);

CREATE TABLE IF NOT EXISTS attribute_option (
id UUID PRIMARY KEY,
attribute_id UUID NOT NULL,
value TEXT NOT NULL,
FOREIGN KEY (attribute_id) REFERENCES attribute(id) ON DELETE CASCADE,
UNIQUE (attribute_id, value)
);

CREATE TABLE IF NOT EXISTS category_attribute (
category_name TEXT NOT NULL,
attribute_id UUID NOT NULL,
PRIMARY KEY (category_name, attribute_id),
FOREIGN KEY (category_name) REFERENCES plant_category(name) ON DELETE
    CASCADE,
FOREIGN KEY (attribute_id) REFERENCES attribute(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS plant (
id UUID PRIMARY KEY,
name TEXT NOT NULL,
latin_name TEXT NOT NULL,
description TEXT NOT NULL,
category_name TEXT,
main_photo_id UUID,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
```

```
FOREIGN KEY (category_name) REFERENCES plant_category(name),
FOREIGN KEY (main_photo_id) REFERENCES file(id)
);

CREATE TABLE IF NOT EXISTS plant_attribute_value (
plant_id UUID NOT NULL,
attribute_id UUID NOT NULL,
float_value FLOAT,
number_value NUMERIC,
text_value TEXT,
FOREIGN KEY (plant_id) REFERENCES plant(id) ON DELETE CASCADE,
FOREIGN KEY (attribute_id) REFERENCES attribute(id) ON DELETE CASCADE,
PRIMARY KEY (plant_id, attribute_id),
CONSTRAINT value_check CHECK (
(float_value IS NOT NULL AND number_value IS NULL AND text_value IS NULL)
   OR
(float_value IS NULL AND number_value IS NOT NULL AND text_value IS NULL)
   OR
(float_value IS NULL AND number_value IS NULL AND text_value IS NOT NULL))
);
```

Таблица 4.2 — Результаты исследования зависимости времени выполнения запроса от схемы данных.

Объем данных	**Время выполнения EAV, мс**	**Время выполнения JSON, мс**	**Отношение JSON к EAV**
Тип запроса: **all**			
100000	946.37	283.25	0.30
200000	1439.67	562.58	0.39
300000	2483.49	870.79	0.35
400000	2216.86	359.21	0.16
500000	3017.53	467.54	0.15
750000	4216.71	684.16	0.16
1000000	6799.15	987.72	0.15
1500000	11596.03	2995.80	0.26
2000000	19885.25	3599.14	0.18
Тип запроса: **category_filter**			
100000	489.25	169.46	0.35
200000	979.83	351.56	0.36
300000	1504.32	538.59	0.36

Таблица 4.2 — Результаты исследования зависимости времени выполнения запроса от схемы данных (продолжение).

Объем данных	**Время выполнения EAV, мс**	**Время выполнения JSON, мс**	**Отношение JSON к EAV**
400000	1401.90	259.97	0.19
500000	1733.73	270.46	0.16
750000	2810.36	430.02	0.15
1000000	4895.13	724.75	0.15
1500000	5680.89	1208.82	0.21
2000000	10253.21	1852.63	0.18
Тип запроса: **height_m_and_diameter_m_filter_**			
100000	951.80	236.99	0.25
200000	1440.97	355.22	0.25
300000	2501.52	747.03	0.30
400000	2193.72	636.89	0.29
500000	2982.99	777.64	0.26
750000	4255.40	1126.81	0.26
1000000	7615.72	2372.24	0.31
1500000	9110.91	2395.17	0.26
2000000	19456.92	5360.59	0.28
Тип запроса: **height_m_filter**			
100000	934.59	375.93	0.40
200000	1438.67	948.76	0.66
300000	2449.28	964.02	0.39
400000	2211.85	627.02	0.28
500000	3006.49	770.74	0.26
750000	4229.37	725.98	0.17
1000000	6992.55	1134.66	0.16
1500000	9511.57	3220.71	0.34
2000000	19410.76	5083.69	0.26
Тип запроса: lot_of_filters			
100000	120.81	54.48	0.45
200000	96.70	104.07	1.08
300000	143.48	157.93	1.10
400000	142.93	154.47	1.08
500000	175.99	191.68	1.09
750000	251.10	272.07	1.08

Таблица 4.2 — Результаты исследования зависимости времени выполнения запроса от схемы данных (продолжение).

Объем данных	**Время выполнения EAV, мс**	**Время выполнения JSON, мс**	**Отношение JSON к EAV**
1000000	445.82	471.85	1.06
1500000	526.31	559.02	1.06
2000000	786.84	845.24	1.07
Тип запроса: string_description_filter			
100000	120.48	131.45	1.09
200000	96.71	101.71	1.05
300000	144.79	155.79	1.08
400000	143.36	153.97	1.07
500000	179.24	193.68	1.08
750000	252.49	272.65	1.08
1000000	430.94	442.67	1.03
1500000	528.20	536.56	1.02
2000000	792.52	847.64	1.07

Приложение Б

Презентация к курсовой работе состоит из 14 слайдов.